

Module 4: Lesson 3

Backpropagation in RNNs



Outline

- ▶ Backprop in RNNs: Clipping gradients
- ▶ Dealing with backprop through time
- ▶ Backprop for RNNs in detail

Backprop in RNNs: Clipping gradients

Backpropagation in recurrent neural networks (RNNs) poses a bunch of challenges. We will use this lesson to identify them and offer some simple (and not so simple) ways to mitigate them.

- ▶ The length of the sequence in RNNs introduces a new dimension for our deep neural networks.
- ▶ This means that in each iteration in backpropagation we have to calculate different gradients through time.
- ▶ The consequence is a potentially huge chain of matrix-products that may result in numerical instability.
- ▶ The latter will cause something that, unfortunately, we are already familiar with: **vanishing and exploding gradients**.

A usual 'hack' used in practice to avoid the vanishing/exploding gradients problem is **gradient clipping**:

- Is essentially a way to force gradients to take smaller values.
- In each update of the gradient: $x \leftarrow x - \eta g$, g will be 'clipped' by:

$$g \leftarrow \min \left(1, \frac{\theta}{\|g\|} \right) g$$

Dealing with backprop through time

Applying backpropagation in RNNs is called **backpropagation through time**. Suppose we are in a simple RNN where hidden and output layers at time t are defined as:

$$h_t = f(x_t, h_{t-1}, w_h) \quad \text{and} \quad o_t = g(h_t, w_o)$$

The loss function across T time steps is:

$$L(x_1, \dots, x_T, y_1, \dots, y_T, w_h, w_o) = \frac{1}{T} \sum_{t=1}^T l(y_t, o_t)$$

Backprop requires us to compute several gradients, for which we can apply the chain rule:

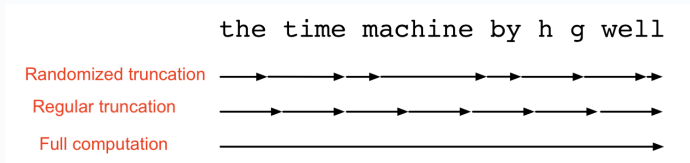
$$\frac{\partial L}{\partial w_h} = \frac{1}{T} \sum_{t=1}^T \frac{\partial l(y_t, o_t)}{\partial w_h} = \frac{1}{T} \sum_{t=1}^T \frac{\partial l(y_t, o_t)}{\partial o_t} \frac{\partial g(h_t, w_o)}{\partial h_t} \frac{\partial h_t}{\partial w_h}$$
$$\frac{\partial h_t}{\partial w_h} = \frac{\partial f(x_t, h_{t-1}, w_h)}{\partial w_h} + \sum_{i=1}^{t-1} \left(\prod_{j=i+1}^t \frac{\partial f(x_j, h_{j-1}, w_h)}{\partial h_{j-1}} \right) \frac{\partial f(x_i, h_{i-1}, w_h)}{\partial w_h}$$

While the chain rule is very useful for the last computation, it can get very long as t increases!

Dealing with backprop through time

There are a bunch of strategies that we can use to deal with the computation of gradients and chain rules when t is very large:

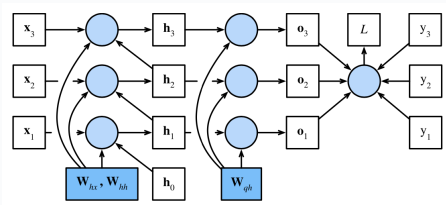
- ▶ Full computation
→ Compute 'the full sum', being aware that things can 'blow up' if t is large
- ▶ Truncating time steps
→ Truncate the sum after τ steps
- ▶ Randomized truncation
→ Use a random variable, which is correct in expectation but truncates the sequence



Backprop for RNNs in detail

The detailed calculations embedded in backprop for RNNs is described in detail in section 9.7.2 in chapter 9 of [Zhang, Ashton, et al. *Dive into Deep Learning*](#).

- ▶ We direct you to that section (part of the required readings) for the full explanation.
- ▶ Here, we simply review a few of the steps part of the process:



Even in this very simply network we will have to, using the chain rule, calculate (in backprop order):

$$\frac{\partial L}{\partial o_t} \rightarrow \frac{\partial L}{\partial W_{qh}} \rightarrow \frac{\partial L}{\partial h_t} \rightarrow \frac{\partial L}{\partial W_{hx}} \rightarrow \frac{\partial L}{\partial W_{hh}} \rightarrow \dots$$

Summary of Lesson 3

In Lesson 3, we have:

- ▶ Identified the sources of vanishing/exploding gradient problems in RNNs
- ▶ Presented several options to deal with long sequences in RNNs
- ▶ Gone through backprop through time in detail

⇒ **References for this lesson:**

Chapter 9, Sections 9.5.3 and 9.7 of [Zhang, Ashton, et al. *Dive into Deep Learning*](#).

TO DO NEXT: This lesson does not have an associated Jupyter Notebook. Please advance to the next lesson.

In the next lesson, we will improve different aspects of the RNN network and revisit the example from Lesson 2, extending it to the multivariate case.